Simulation-Grouped Per-Class QoS Prediction for SD-WAN

Radhika (Rishiv) Shitlani
School of Computer Science
University of Galway, Galway, Ireland

Abstract

Software-defined wide area networks (SD-WANs) need to treat traffic classes differently, yet a single strong global prediction score can quietly hide poor performance for exactly the best-effort class that needs the most scrutiny. This paper studies per-flow delay prediction and service-level-agreement (SLA) violation detection using 29,280 flow records from 400 BNNetSimulator runs spanning Gold, Silver, and Bronze traffic under three scheduling policies. An XGBoost regressor is evaluated with five-fold cross-validation grouped by simulation identifier, so that no simulation run contributes flows to both the training and validation side of a fold. Under this grouped protocol, the model reaches an overall log-delay R^2 of 0.747, with class-specific values of 0.907, 0.898, and 0.680 for Gold, Silver, and Bronze; the corresponding SLA-violation F1 scores are 0.940, 0.680, and 0.554. A conventional row-wise split reports a higher overall R^2 of 0.766, which turns out to be optimism rather than genuine accuracy, caused by sharing simulation context across folds. The same grouped protocol shows that 90th-percentile delay is only moderately predictable ($R^2 = 0.293$ overall) and that jitter is not predictable at all ($R^2 = -0.040$). Taken together, the results argue that grouped, per-class evaluation is not an optional refinement but a precondition for trusting any QoS prediction claim in SD-WAN.

Index Terms

SD-WAN, quality of service, machine learning, XGBoost, grouped cross-validation, SLA prediction

I. Introduction

Software-defined wide area networking (SD-WAN) separates network control from forwarding, giving operators centralized control over routing and QoS policy across heterogeneous links. In practice, however, most of that control is still exercised too late: an operator changes queue weights or reroutes traffic only after congestion has already affected users. Predicting QoS ahead of an SLA breach could turn this reactive habit into something proactive, but only if the prediction is trustworthy for the traffic that actually needs protecting. Gold, Silver, and Bronze traffic differ in priority, in how their delay behaves, and in what an error actually costs, so a model that looks accurate on average can still be quietly unreliable for exactly the class most exposed to congestion.

Machine learning has already found several footholds in SD-WAN research: predicting how many virtual network functions a network will need [1], routing

traffic based on application type [2], and steering traffic engineering with reinforcement learning [3], [4]. Other work optimises SD-WAN routes and queueing policies directly from measured QoS [5], [6]. What these studies have in common is that they show predictive or adaptive control works in principle, but none of them directly answers a narrower, more practical question: how accurately can continuous delay and SLA violations actually be predicted, separately for each traffic class, from a compact and reproducible feature set?

A second, easier-to-overlook problem is evaluation leakage. Every packet-level simulation run produces many flow rows at once, so randomly splitting individual rows can easily place flows from the same run on both sides of a train/validation split. The simulation identifier itself is never used as an input feature, but the shared topology, load, and scheduler conditions behind those rows can still make the validation task artificially easy. This paper treats the simulation run, not the individual flow row, as the unit that gets split.

The contributions are threefold:

- a reproducible per-class delay-prediction study using 29,280 flows from 400 BNNetSimulator runs;
- a simulation-grouped evaluation reporting both regression and SLA-trigger performance separately for Gold, Silver, and Bronze traffic, with row-wise cross-validation retained only as a sensitivity comparison; and
- an extension of the grouped protocol to two secondary QoS targets, tail (90th-percentile) delay and jitter, establishing the boundary of what a static configuration feature set can and cannot predict.

II. Related Work

Machine learning has been applied to several adjacent network-management tasks, though rarely to per-class QoS prediction itself. Rahman *et al.* predict how many virtual network functions a system will need as load changes [1], while Zheng *et al.* combine traffic classification with QoS-aware routing decisions [2]; both use machine learning to inform a decision rather than to predict a continuous QoS outcome per class. For SD-WAN specifically, Quang *et al.* formulate joint routing and QoS policy optimisation [5] and later add passive available-bandwidth estimation to a two-loop controller [6]; these are optimisation systems built around a fixed queueing model, not learned predictors evaluated against ground truth. Recent work also explores safe deep reinforcement learning for load balancing [3] and multi-objective, graph-based traffic engineering [4]. None of these studies reports how well delay or SLA compliance can be predicted separately for each priority class, which is the specific gap this paper addresses.

When labelled enterprise telemetry is not available, network simulators offer a reproducible alternative. BNNetSimulator generates packet-level data compatible with the modelling methodology behind RouteNet-Fermi [7], [8], and this study uses it to ask a narrower, more practical question than the wider literature does: can static configuration and path features alone support reliable, class-specific QoS prediction? XGBoost is used as the main model because gradient-boosted trees remain strong on small-to-medium tabular datasets [9], [10], and because, unlike a neural network, it trains quickly and its feature importances can be read directly.

III. Methodology

A. Dataset and QoS Classes

The dataset was generated with the official BNNetSimulator container [7] and contains 29,280 source-destination flow records from 400 simulation runs, with 30–132 flows per run. Traffic is split into three classes: Gold (4,592 rows, highest priority), Silver (8,991 rows, interactive traffic), and Bronze (15,697 rows, best effort). The experiments span strict priority, WFQ, and deficit round robin scheduling across four scenarios: fixed WFQ weights, randomised WFQ profiles, and two mixed-policy scenarios with skewed and equal class proportions. Traffic loads are kept light to moderate throughout, so the results below should be read as a controlled study of per-class prediction, not a stress test of a saturated production network.

Each input row carries the traffic class, offered bandwidth, traffic distribution, route length, network size, maximum average load, link bandwidth, scheduling policy, class queue weight, and minimum class weight. The prediction target is the natural logarithm of average delay in seconds. Everything that would leak the answer (raw delay, jitter, packet loss, delay percentiles, achieved bandwidth, the scenario label, and the simulation identifier itself) is excluded from the feature matrix. Missing numeric values are median-imputed and categorical variables are one-hot encoded, both done separately inside each training fold to avoid leakage.

B. Prediction and Grouped Evaluation

The primary regressor is XGBoost, with 200 trees, depth four, a learning rate of 0.05, and row and feature subsampling both set to 0.9. Because the model is trained on log delay, predictions are converted back to milliseconds before being reported, so that the error numbers mean something operationally. Performance

is measured with mean absolute error (MAE) on the millisecond scale and R^2 on log delay.

Five-fold GroupKFold cross-validation is used with *simulation_id* as the grouping key, so that every simulation run's flows end up entirely on one side of each fold, never split across training and validation. As a sensitivity check, the same model is also evaluated with a conventional shuffled, row-wise five-fold split, so the two protocols can be compared directly. Preprocessing and model fitting are refit independently inside every fold.

Continuous delay predictions are also turned into binary SLA-violation triggers, using thresholds of 30, 50, and 60 ms for Gold, Silver, and Bronze respectively. Precision here tells us how much a trigger can be trusted once it fires, recall tells us how many real violations were actually caught, and F1 balances the two. The Bronze threshold came out of an exploratory sensitivity sweep rather than a fixed requirement, so it should be read as a reasonable operating point rather than a claimed universal SLA.

The same grouped folds and leakage-safe feature exclusions are reused for two secondary targets, 90th-percentile delay and jitter, evaluated directly in milliseconds since that is the scale the operational question actually cares about. A negative R^2 here is not a bug: it simply means the model's predictions generalise worse than just guessing the validation-fold mean.

C. Reproducibility

Data generation, preprocessing, grouped evaluation, and plotting are implemented as separate, reusable Python scripts. Randomised estimators use a fixed seed (42), and every reported metric comes from out-of-fold predictions rather than training predictions, so nothing here is inflated by the model having already seen the row it is being scored on. The grouped evaluator also records which fold unit (row or simulation run) produced each result, to avoid the two being mixed up later during reporting. The generated dataset, experiment scripts, and result tables are all available in the project repository; a permanent archival identifier will be added for the camera-ready version.

IV. Results

A. Per-Class Delay and SLA Prediction

Table I reports the primary grouped results. Overall R^2 is 0.747, with a delay-scale MAE of 13.19 ms, but that single number hides a large class-dependent split. Gold and Silver are predicted accurately ($R^2 = 0.907$ and 0.898), while Bronze falls to 0.680 and its MAE nearly triples to 21.57 ms. This pattern makes

TABLE I
FIVE-FOLD SIMULATION-GROUPED RESULTS

Class	MAE (ms)	R^2	Prec.	Recall	F1
Gold	3.17	0.907	0.986	0.898	0.940
Silver	3.67	0.898	0.798	0.593	0.680
Bronze	21.57	0.680	0.569	0.540	0.554

TABLE II
SENSITIVITY TO THE CROSS-VALIDATION UNIT

Protocol	Overall R^2	Gold R^2	Silver R^2	Bronze R^2
Row K-fold	0.766	0.914	0.908	0.704
Grouped by run	0.747	0.907	0.898	0.680

operational sense: protected traffic is shaped mostly by stable topology and scheduler configuration, exactly what the static features describe, while best-effort delay is driven by transient contention that those same features cannot see.

Gold's SLA trigger keeps its precision very high at 0.986, meaning a controller acting on a predicted Gold violation is almost never wrong to do so; this matters because false positives on the highest-priority class would trigger unnecessary rerouting. Silver and Bronze are noticeably less reliable, especially on recall, and Bronze's F1 of 0.554 makes an important point on its own: a regressor that looks perfectly usable in aggregate does not automatically translate into a strong SLA-violation detector for best-effort traffic.

B. Effect of Grouping by Simulation

Table II and Fig. 1 compare the grouped protocol against a conventional shuffled row-wise split. Grouping brings overall R^2 down from 0.766 to 0.747, and the biggest single drop is Bronze, from 0.704 to 0.680, with its SLA F1 falling from 0.592 to 0.554; Gold barely moves. In other words, the leakage from row-wise splitting is real but modest, not catastrophic, yet it is large enough to change what a paper can honestly claim about generalisation, which is reason enough to treat the simulation run, not the row, as the correct unit for evaluation.

An exploratory row-wise comparison found near-identical global scores for XGBoost ($R^2 = 0.766$), a multilayer perceptron (0.766), and an FT-Transformer (0.762). These neural results have not yet been repeated under simulation-grouped folds, so they are reported here only as supporting evidence, not as

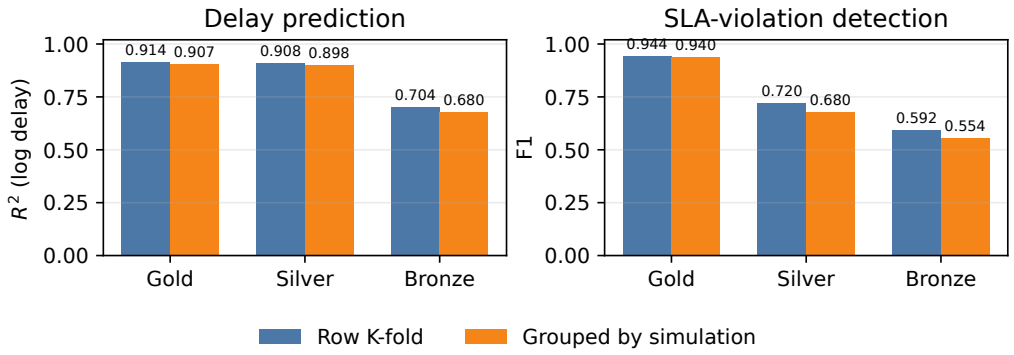

Fig. 1. Row-wise and simulation-grouped performance by QoS class.

TABLE III
EXPLORATORY ROW-WISE MODEL COMPARISON

Model	Overall R^2	Gold R^2	Silver R^2	Bronze R^2
XGBoost	0.766	0.914	0.908	0.704
MLP	0.766	0.912	0.908	0.705
FT-Transformer	0.762	0.900	0.898	0.703

the paper's primary generalisation estimate, but they are enough to justify XGBoost as the lower-cost model of choice for the rest of the study.

C. Model Complexity and Feature Evidence

Table III breaks that exploratory comparison down by class. All three model families land on essentially the same Bronze ceiling under the row-wise protocol, and the FT-Transformer is only marginally behind on Gold and Silver; no gap exceeds 0.014 R^2. That is not proof of equivalence under grouped validation, but it is a clear signal: extra neural complexity buys nothing here, because it cannot compensate for short-timescale traffic state the features simply do not contain.

SHAP values for the final XGBoost model back this up (Fig. 2). Route length dominates: more hops push predictions toward higher log delay, and shorter paths pull them down. Network size and traffic class follow some way behind, with queue weights and scheduling-policy indicators contributing comparatively little on their own. These are associations learned within the simulator, not causal estimates, but they tell a consistent story: the model leans almost entirely on stable path structure. That structure is genuinely informative for protected Gold and Silver flows, but it simply does not describe the transient contention that drives Bronze delay.

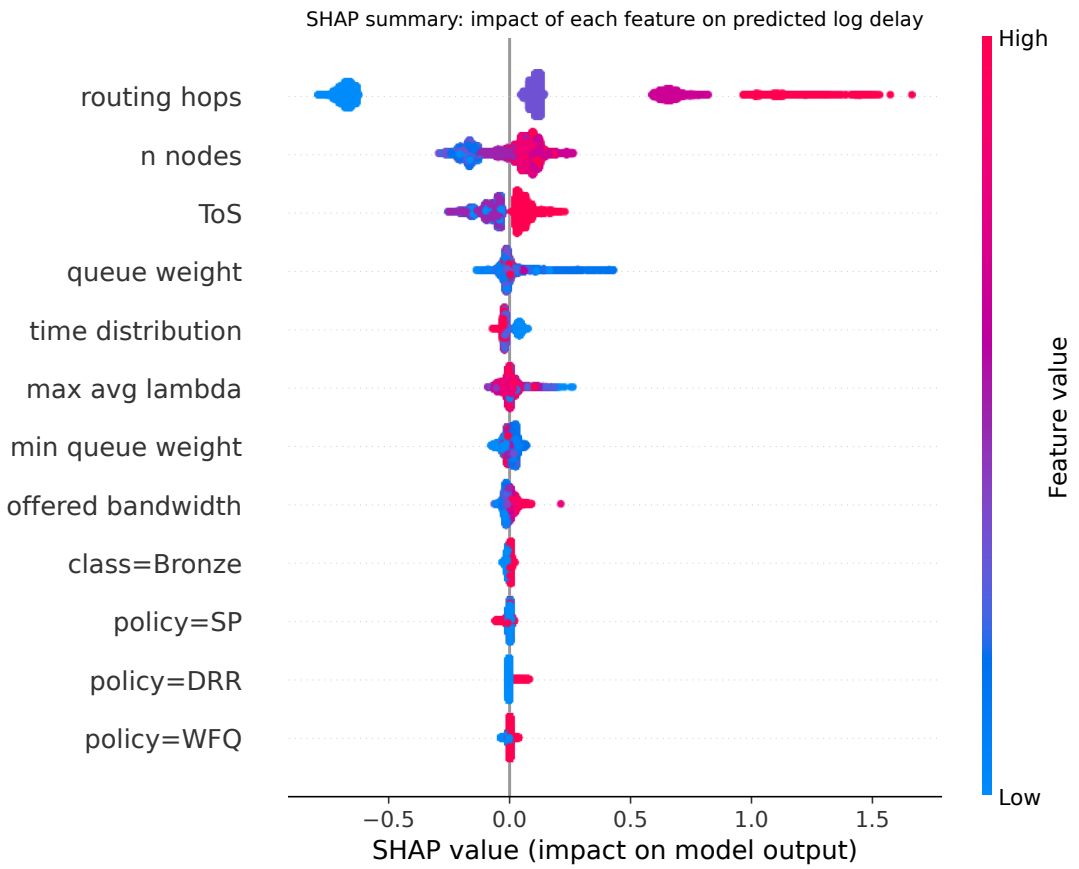

Fig. 2. SHAP summary for the XGBoost log-delay model; colour denotes feature value.

D. Tail Delay and Jitter

Table IV applies the same grouped protocol to two secondary QoS targets. Tail delay is noticeably harder to predict than average delay: overall R^2 drops to 0.293, reaching only 0.550 and 0.490 for Gold and Silver and falling further to 0.269 for Bronze, whose MAE climbs to 51.81 ms, consistent with rare contention events dominating the upper tail.

Jitter does not generalise from the static feature set at all: overall R^2 is -0.040, and every class comes out negative. Gold and Silver have small absolute MAE, but their within-class variance is so low that even modest errors end up worse than simply predicting the validation-fold mean. Together these results sharpen the boundary of what this feature set can actually tell us: it describes mean delay well, tail delay only partly, and moment-to-moment delay variation not at all.

TABLE IV
GROUPED RESULTS FOR SECONDARY QOS TARGETS

Target	Class	MAE (ms)	R^2
Delay p90	Overall	32.51	0.293
	Gold	8.92	0.550
	Silver	10.87	0.490
	Bronze	51.81	0.269
Jitter	Overall	3.22	−0.040
	Gold	0.51	−7.555
	Silver	1.01	−13.182
	Bronze	5.28	−0.034

V. Discussion and Limitations

The central finding here is not just that delay is predictable: it is that how predictable it is depends on the traffic class, the target, and the validation design chosen. Route length, network size, and traffic class encode stable structure, and that structure explains protected mean delay well. Bronze has the lowest service priority and is therefore more exposed to unobserved bursts and queue occupancy the model never sees. The collapse on jitter drives the point home: no amount of model complexity can recover information that simply is not in the features.

Several limitations should temper these claims. First, the data is simulator-generated and covers only a light-to-moderate load regime; grouped folds test generalisation to unseen runs from the same generator, not transfer to a real enterprise SD-WAN. Second, the feature set is entirely static: telemetry such as instantaneous queue occupancy, arrival-rate windows, and recent loss would be needed to model transient behaviour, and none of that is present here. Third, the SLA thresholds used are experimental operating points, not contractual values pulled from a real deployment. Future work should repeat every model comparison under grouped outer folds, test under heavier congestion, and validate against real controller telemetry.

VI. Conclusion

This paper presented a simulation-grouped, per-class evaluation of QoS prediction for SD-WAN-like networks. XGBoost generalises well to unseen simulation runs for Gold and Silver mean delay, is noticeably less reliable for Bronze, and does not generalise for jitter at all. Row-wise cross-validation overstates how well the model actually performs, which confirms that the simulation run, not the individual flow, has to be the unit of evaluation. The broader point is a simple one: trustworthy AI-assisted QoS prediction needs class-aware metrics,

group-safe validation, and an honest account of what static network features can and cannot support.

References

[1] S. Rahman, T. Ahmed, M. Huynh, M. Tornatore, and B. Mukherjee, "Auto-scaling vnfs using machine learning to improve qos and reduce cost," in *Proc. IEEE Int. Conf. Commun. (ICC)*, 2018, pp. 1–6.

[2] W. Zheng, M. Yang, C. Zhang, Y. Zheng, Y. Wu, Y. Zhang, and J. Li, "Application-aware qos routing in sdns using machine learning techniques," *Peer-to-Peer Networking and Applications*, vol. 15, no. 1, pp. 529–548, 2022.

[3] L. Dinh, P. T. A. Quang, and J. Leguay, "Towards safe load balancing based on control barrier functions and deep reinforcement learning," *arXiv preprint arXiv:2401.05525*, 2024.

[4] Y. Chen, A. Gu, L. Cui, and L. Lin, "MTEAL: Network routing optimization of SD-WAN traffic engineering integrating multi-dimensional QoS metrics," *Journal of Network and Computer Applications*, vol. 242, p. 104272, 2025.

[5] P. T. A. Quang, X. Gong, C. Liu, K. Li, J. Leguay, X. Zhang, Y. Zhang, J. Li, and K. Ye, "Routing and qos policy optimization in sd-wan," *arXiv preprint arXiv:2209.12515*, 2022.

[6] P. T. A. Quang, J. Leguay, X. Gong, and H. Xu, "Global qos policy optimization in sd-wan," in *Proc. IEEE 9th Int. Conf. Network Softwarization (NetSoft)*, 2023, pp. 202–206.

[7] BNN-UPC, "BNNetSimulator: A packet-level network simulator to generate datasets for research and analysis," GitHub repository: https://github.com/BNN-UPC/BNNetSimulator, 2023.

[8] M. Ferriol-Galmés, J. Paillisse, J. Suárez-Varela, K. Rusek, S. Xiao, X. Shi, X. Cheng, P. Barlet-Ros, and A. Cabellos-Aparicio, "Routenet-fermi: Network modeling with graph neural networks," *IEEE/ACM Transactions on Networking*, vol. 31, no. 6, pp. 3080–3095, 2023.

[9] L. Grinsztajn, E. Oyallon, and G. Varoquaux, "Why do tree-based models still outperform deep learning on typical tabular data?" in *Advances in Neural Information Processing Systems*, vol. 35, 2022.

[10] R. Shwartz-Ziv and A. Armon, "Tabular data: Deep learning is not all you need," *Information Fusion*, vol. 81, pp. 84–90, 2022.